

3 Real Production Incidents · Root Cause · Detection · Fix

Scenario 1

Kubernetes – Conntrack Table Exhaustion

Scenario 2

Terraform – Silent State Drift Destroys Production DB

Scenario 3

AWS ALB – TCP Keepalive Mismatch causing 502 Errors

Each scenario covers: What happened · Why it was hard to spot · How a senior found it · How to fix it · How to prevent it

SCENARIO 1

Kubernetes: Random Timeouts

Pod traffic is failing — but everything looks perfectly healthy

What is this scenario about?

Imagine you run a shopping website on Kubernetes. At 2 AM, customers start seeing slow or failed requests. Your team wakes up, checks everything — pods are running, CPU is normal, memory is fine, no one deployed anything. Yet 20% of requests are failing with timeout errors. This is the nightmare scenario: the system lies to you.

Environment (What was running)

- EKS — Kubernetes cluster managed by AWS
- Istio — a service mesh that controls traffic between services
- Java microservices — the actual application code
- HPA — Horizontal Pod Autoscaler (adds/removes pods based on load)
- Prometheus + Grafana — monitoring and dashboards
- NodeLocal DNSCache — speeds up DNS lookups inside the cluster

What is Kubernetes? (Beginner Explanation)

Think of Kubernetes as a smart manager for your applications. Instead of running your app on one computer, Kubernetes runs many small copies of it (called **pods**) across many computers (called **nodes**). If traffic increases, it adds more pods. If a pod crashes, it restarts it. The goal is: your app is always running and fast.

What Symptoms Were Visible?

- **API latency jumped: 200ms → 8 seconds**
Responses became 40x slower. Users experienced freezing or timeout screens.
- **Only 20% of requests failed**
Not everything was broken — which made it confusing. Some requests worked fine.
- **CPU and Memory: completely normal**
Every monitoring dashboard looked green. No obvious resource pressure.
- **No pod restarts, no crashes**
kubectl get pods showed everything as 'Running'. No red flags.



No recent deployment

Nobody pushed new code. The app hadn't changed.



Database healthy, nodes healthy

Every infrastructure component reported as working fine.

Why Was This So Misleading?

Every standard check said: *everything is fine*. A junior engineer would spend hours restarting pods or blaming the application team. The app team would say 'we didn't change anything' — and they'd be right. The real problem was completely invisible in normal monitoring.

What Was Actually Happening? (Root Cause)

The real villain was something called the **Linux Conntrack Table**. Here's what that means in simple terms:

What is the Conntrack Table? (Plain English)

- Every time two computers talk over the network, Linux tracks that conversation.
- It uses a table called 'nf_conntrack' to remember each active connection.
- This table has a maximum size — like a notebook with limited pages.
- When the table is FULL, Linux silently drops new network packets.
- No error message. No log. It just drops them — like tearing pages out silently.

The Chain of Events (Step by Step)

1

NodeLocal DNSCache crashed on some nodes

This is a component that speeds up DNS (the system that converts names like 'myservice' into IP addresses). When it crashed, it stopped working.

2

Pods couldn't use the fast DNS cache anymore

Every pod on that node had to send DNS queries the slow way — directly to CoreDNS (the cluster's central DNS server).

3

DNS retries exploded

When a DNS query timed out or failed, the system retried. Each retry created a new entry in the conntrack table.

4

Conntrack table filled up

Thousands of retry entries flooded the table until it was completely full.

5

Linux started silently dropping packets

New connection attempts got dropped at the kernel level. The application had no idea — it just saw 'timeout'.



Istio retries partially hid the problem

Istio automatically retried failed requests. So some requests eventually succeeded, making only 20% fail — confusing the investigation further.

Why Only SOME Pods Failed?

The conntrack table is per-node (per-computer). The DNS cache crashed only on **some nodes**. Pods running on unaffected nodes worked perfectly. Pods on affected nodes saw random failures. This is why the failures looked random and intermittent — they depended on which node your pod was on.

How Did the Senior Engineer Find It?

After ruling out the app, the database, the CPU, and the memory, the senior engineer SSHed directly into one of the affected nodes and ran this command:

```
$ dmesg | grep conntrack
nf_conntrack: table full, dropping packet
nf_conntrack: table full, dropping packet
nf_conntrack: table full, dropping packet
```

That single output revealed everything. The kernel was silently dropping packets because the conntrack table was full. Then they checked the DNS cache and found it was in a crash loop — which had triggered the whole chain.

How to Fix It

Immediate Fix (Right Now)

- SSH into the affected node
- Increase conntrack limit immediately:
- `sysctl -w net.netfilter.nf_conntrack_max=524288`
- Restart the NodeLocal DNSCache pod

Permanent Fix (Long Term)

- Add `conntrack_max` to node config permanently
- Add Prometheus alert for conntrack usage > 80%
- Tune Istio retry settings to reduce retry storms
- Set up DNS cache health monitoring

Key Lesson for You

What a Beginner vs Senior Does Here

- Junior: checks pods, restarts app, blames developer. Stays at application layer.
- Senior: asks 'what layer is lying?' and goes deeper — into OS, kernel, networking.
- The symptom (timeout) was application-level. The cause was kernel-level.
- You MUST understand that healthy pods don't mean healthy nodes.

- Always check: dmesg, conntrack, DNS logs when you see random intermittent failures.

SCENARIO 2

Terraform Wants to Delete Your Production DB

Nobody changed anything — yet Terraform plans to destroy everything

What is this scenario about?

An engineer runs a routine **terraform plan** — a dry-run that shows what changes Terraform wants to make. Instead of seeing no changes, they see the most terrifying message in infrastructure engineering: *'aws_db_instance.prod will be replaced'*. Terraform wants to destroy and recreate the production PostgreSQL database. All data would be lost. Nobody changed the code. Nobody changed the variables. How is this possible?

Environment (What was running)

- Terraform — Infrastructure as Code tool (write code → create AWS resources)
- AWS RDS PostgreSQL — managed database service (your actual production database)
- S3 Backend — Terraform stores its 'memory' (state file) in an S3 bucket
- Multiple workspaces — separate environments like dev, staging, production

What is Terraform? (Beginner Explanation)

Terraform is a tool that lets you describe your infrastructure in code. Instead of clicking around in the AWS Console to create a database, you write a file that says: *create a PostgreSQL database with these settings*. Terraform reads that file and creates the real AWS resource for you.

The Three Things Terraform Tracks

- Your CODE — what you wrote in .tf files (desired state)
- The STATE FILE — Terraform's memory of what it created (stored in S3)
- REAL AWS — what actually exists in your AWS account right now

What Symptoms Were Visible?



terraform plan showed: DB will be REPLACED

The plan wanted to delete the production database and create a new one. This would cause complete data loss and hours of downtime.



No code changes in Git history

The engineer searched through all recent commits. Nobody changed anything related to the database configuration.



No variable changes

All Terraform variables were the same as before.



Happened suddenly after a routine plan

The previous week's plan was clean. This week: disaster.

Why Was This So Misleading?

The team spent over an hour checking Git blame, looking at who ran what command, checking for accidental edits. Everything pointed to 'Terraform is broken' or 'the state is corrupted'. Neither was true.

What Was Actually Happening? (Root Cause)

The real cause happened in **two separate events, months apart**. This is what makes it so hard:

1

Months ago: Someone enabled encryption via the AWS Console



A DevOps engineer logged into the AWS Console and manually toggled 'storage_encrypted = true' on the RDS instance. A quick fix. Nobody updated the Terraform code.

2

The Terraform state still said: encrypted = false



Terraform's memory (state file) was never updated. It still believed the database was unencrypted. The real DB and Terraform's memory diverged.

3

The old Terraform AWS provider ignored this drift



For months, when Terraform ran, the old provider version didn't properly detect that the encryption setting had changed. So no one noticed the mismatch.

4

Provider upgrade changed the detection behavior



Someone upgraded the Terraform AWS provider to a newer version. The new version correctly detected: 'Wait — the real DB says encrypted=true, but the state says false. That's an immutable attribute. Must replace.'

5

Terraform planned a destroy + recreate



Because encryption on RDS cannot be changed in-place (it's immutable), Terraform concluded the only way to fix the mismatch was to delete the DB and create a new one with the correct settings.

What is 'Immutable' in AWS?

- Some AWS resource settings cannot be changed after creation — they are 'immutable'.
- storage_encrypted on RDS is one such setting.
- If Terraform detects a drift on an immutable setting, it MUST destroy and recreate.
- This is why the fix is: update the state to match reality — NOT destroy the DB.

How Did the Senior Engineer Find It?

The senior engineer stopped looking at Terraform code and started asking: *'When did the real database and Terraform's state start to differ?'* They compared the state file vs the actual AWS RDS settings:

```
# Check what Terraform thinks the DB looks like $ terraform state show  
aws_db_instance.prod storage_encrypted = false <-- Terraform memory # Check  
what AWS actually has $ aws rds describe-db-instances --db-instance-identifier  
prod StorageEncrypted: true <-- Real AWS
```

That mismatch — false vs true — was the entire problem. Then they checked CloudTrail (AWS audit logs) and found the manual change made months ago by a team member in the console.

How to Fix It (Without Destroying the Database)

```
# Step 1: Remove the DB from Terraform's memory (does NOT delete the real DB) $  
terraform state rm aws_db_instance.prod # Step 2: Re-import the real DB into  
Terraform's state $ terraform import aws_db_instance.prod your-db-identifier #  
Step 3: Update your .tf file to match reality # Change: storage_encrypted =  
false # To: storage_encrypted = true # Step 4: Verify plan shows no changes $  
terraform plan # Expected: No changes. Infrastructure is up-to-date.
```

What Went Wrong

- Never change AWS resources manually via Console
- All changes must go through Terraform code
- State file is now stale and dangerous

How to Prevent This

- Use AWS Config rules to detect manual drift
- Set up Terraform drift detection in CI/CD
- Lock down Console permissions to prevent manual changes
- Use terraform plan regularly, not just on deploy

Key Lesson for You

The Golden Rule of Terraform

- Terraform code = what you WANT to exist
- Terraform state = what Terraform REMEMBERS creating
- AWS reality = what ACTUALLY exists right now
- When these three diverge, Terraform can plan catastrophic changes.
- NEVER change AWS resources manually if Terraform manages them.
- If you must make an emergency manual change, update the state file immediately.

SCENARIO 3

AWS ALB: Random 502 Errors

Everything is healthy — but users keep getting 502 Bad Gateway

What is this scenario about?

Users randomly see **502 Bad Gateway** errors on your website. You check everything: EC2 instances are running, the application responds locally, health checks pass, security groups are correct. Yet the ALB (load balancer) keeps returning 502 to real users. The errors are intermittent — sometimes they happen, sometimes they don't. This is one of the hardest classes of bugs: a networking behavior mismatch that only appears under specific timing conditions.

Environment (What was running)

- ALB (Application Load Balancer) — distributes user traffic to EC2 instances
- Auto Scaling Group — automatically adds/removes EC2 instances based on load
- EC2 instances — servers running your application
- Route53 — DNS (maps your domain name to the ALB's address)
- Multi-AZ — running in multiple AWS availability zones for high availability

What is an ALB? (Beginner Explanation)

An Application Load Balancer sits in front of your servers. When a user visits your website, they hit the ALB first. The ALB decides which server (EC2 instance) should handle that request and forwards it there. This spreads traffic evenly across many servers.

What Symptoms Were Visible?

- **502 Bad Gateway — intermittently from ALB**
Users randomly got error pages. Refreshing sometimes worked. This intermittency is the key clue.
- **EC2 instances: healthy**
All instances showed as running. No crashes, no high CPU.
- **curl localhost:8080 on the instance: worked perfectly**
When engineers SSHed in and tested the app locally, it responded fine.
- **ALB health checks: passing**
The ALB's automatic health checks were all green.

■ Security groups and NACLs: correct

All firewall rules were properly configured.

■ No recent deployments or config changes

Nothing had been changed recently. The 502s just started appearing.

Understanding TCP Connections (Plain English)

To understand this bug, you need to understand how the ALB and your server communicate. They use **TCP connections** — think of it like a phone call.

TCP Connection Analogy

- When the ALB wants to send a request to EC2, it opens a 'phone call' (TCP connection).
- Instead of hanging up after each request (expensive!), both sides keep the line open.
- This is called HTTP Keep-Alive — reusing the same connection for multiple requests.
- The ALB has an 'idle timeout': if nobody talks for 60 seconds, it hangs up.
- Your app also has a keepalive timeout: it decides when to hang up on its end.

What Was Actually Happening? (Root Cause)

The application team set HTTP keepalive timeout to **5 seconds**. The ALB idle timeout was **60 seconds**. This mismatch created a race condition:

1 ■ ALB opens a connection to EC2 and sends a request

The request is handled successfully. Both sides keep the connection open for reuse. The ALB thinks: 'I can reuse this for 60 seconds.'

2 ■ 5 seconds of silence pass

The app's keepalive timeout (5s) expires. From the app's perspective, the connection is dead. It closes its end of the socket. But it doesn't loudly announce this to the ALB.

3 ■ A new user request arrives at the ALB (after 5s, before 60s)

The ALB thinks the old connection is still valid — it's only been 5-60 seconds and the ALB's timeout is 60 seconds. So it reuses the 'dead' connection.

4 ■ ALB sends the new request on the dead connection

The EC2 instance receives data on a socket it already closed. It responds with RST (reset) — essentially saying 'I don't know what you're talking about.'

5 ■ ALB gets RST — returns 502 to the user

The ALB can't deliver the request. It tells the user: 502 Bad Gateway. The user's request was lost.

Why Health Checks Still Passed

- Health checks happen on their OWN connections — they open fresh ones.

- The dead/reused connections were DIFFERENT connections from health check ones.
- So health checks always worked, masking the real problem completely.
- This is why 'health checks green' does NOT mean 'all connections are fine'.

How Did the Senior Engineer Find It?

After checking all the obvious things, the senior engineer captured actual network traffic on the EC2 instance using **tcpdump**:

```
# Capture traffic between ALB and app on port 8080 $ tcpdump -i eth0 -w
capture.pcap port 8080 # Then opened in Wireshark and saw the pattern: # ALB →
EC2: [SYN] - opening connection # EC2 → ALB: [SYN-ACK] - connection accepted #
...request handled successfully... # EC2 → ALB: [FIN] - app closes connection
after 5s keepalive # ALB → EC2: [new request data] - ALB reuses what it thinks
is alive! # EC2 → ALB: [RST] - 'This connection is closed!' # ALB → user: 502
Bad Gateway
```

The RST packets told the whole story. Then they correlated the timing of RST packets with the 5-second app keepalive timeout — perfect match.

How to Fix It

Fix Option 1: Increase App Timeout (Recommended)

- Increase app keepalive timeout to 75 seconds
- This is longer than ALB's 60s idle timeout
- ALB will close idle connections first — before app does
- App never receives requests on dead connections

Fix Option 2: Reduce ALB Timeout

- Reduce ALB idle timeout to less than app keepalive
- Example: ALB = 30s, App keepalive = 45s
- Same principle: ALB closes first
- Less ideal as it reduces connection reuse efficiency

The Golden Rule (Never Forget This)

- Backend keepalive timeout MUST always be GREATER than ALB idle timeout.
- Not equal — GREATER. Equal still causes a race condition.
- Safe formula: Backend keepalive = ALB idle timeout + 15 seconds minimum.
- Example: ALB idle = 60s → App keepalive should be 75s or more.

```
# AWS CLI: set ALB idle timeout to 30 seconds $ aws elbv2
modify-load-balancer-attributes \ --load-balancer-arn \ --attributes
Key=idle_timeout.timeout_seconds,Value=30 # In your Java/Node/Python app, set
keepalive > 30s # Example for Node.js (Express/HTTP): server.keepAliveTimeout =
```

```
45000; // 45 seconds in ms server.headersTimeout = 46000; // must be >
keepAliveTimeout
```

Key Lesson for You

What a Beginner vs Senior Does Here

- Junior: sees 502, checks app logs, restarts instances, gives up.
- Senior: asks 'what is the ALB actually doing at the TCP level?'
- The bug existed at the TCP networking layer — invisible to app logs.
- tcpdump and Wireshark are essential skills for any infrastructure engineer.
- Always check: ALB timeout settings vs backend keepalive settings when you see 502s.
- Intermittent failures = timing/race conditions = think about timeouts first.

Summary: What You Should Take Away

Scenario	Real Cause	Why Hard	Key Command
K8s Timeouts	Conntrack table full due to DNS retry storm	Pods healthy, CPU fine, app-level logs only showed 'timeout'	<code>dmesg grep conntrack</code>
Terraform DB	Months-old manual Console change discovered after provider upgrade	No recent code changes; drift was silent for months	<code>terraform state show terraform import</code>
ALB 502s	Backend keepalive (5s) shorter than ALB idle timeout (60s)	Health checks passed, app worked locally, no logs showed RST	<code>tcpdump -i eth0 port 8080</code>

The Mindset Shift: Junior → Senior

Junior Thinking

- 'Something failed' — stays at symptom layer
- 'Logs show timeout' — takes it at face value
- 'CPU is fine' = system is healthy
- 'Terraform wants to replace' = Terraform is broken
- 'Health checks pass' = ALB is fine
- Restarts the app. Blames developer.

Senior Thinking

- 'Which LAYER is lying to me?'
- 'WHY timeout? What dropped the packet?'
- CPU fine but kernel/network can still fail silently
- 'State drifted. When? Why? Import instead of replace.'
- 'Health checks use fresh connections — not reused ones.'
- SSH into node. Run tcpdump. Check dmesg.

3 Commands Every Infrastructure Engineer Must Know

1. `dmesg | grep -i conntrack` — detect kernel packet drops & conntrack exhaustion (Scenario 1)
2. `tcpdump -i eth0 -w out.pcap port 80` — capture real TCP traffic, find RST packets (Scenario 3)
3. `terraform state show <resource>` — compare Terraform memory vs real AWS (Scenario 2)

*Real production debugging is not about knowing answers instantly — it is about knowing **which layer to question next**. Every senior engineer was once confused by these exact symptoms. The difference is they kept digging deeper.*